

Compiler Design Lab Report 2

Lexical Analysis using Lex/Flex

Abdullah Al Jubaer Gem

ID: 210041226

Section: 2B

May 7, 2026

1 Introduction

The objective of this lab is to implement a Lexical Analyzer using the Lex (Flex) tool. Lex automates the process of generating a scanner by using regular expressions to match tokens in the source code.

2 Token Definitions

The Lex file defines patterns for the following token categories:

- **Keywords:** Reserved identifiers used by the scanner and basic C-like syntax.
include, int, float, if, else, print, while, for, return, main, etc.
- **Constants:**
 - **Integers:** Sequences of digits ($[0-9]^+$).
 - **Floats:** Digit sequences with a decimal point ($[0-9]^+\.[0-9]^+$).
 - **Characters:** Single characters enclosed in single quotes.
 - **Strings:** Byte sequences enclosed in double quotes.
- **Identifiers:** Names starting with a letter or underscore, followed by alphanumeric characters.
 $[A-Za-z_][A-Za-z0-9_]^*$
- **Operators:**
 - **Arithmetic:** +, -, *, /, =
 - **Increment/Decrement:** ++, --
 - **Relational:** ==, !=, <, >, <=, >=
 - **Logical:** &&, ||
 - **Bitwise:** &, |, ^, ~, <<, >>
- **Delimiters:** Brackets, braces, parentheses, and punctuation.
(,), {, }, [,], ;, ,, %, %%

3 Solution Approach

The lexical analyzer is implemented using Lex specifications:

1. **Definitions Section ($\% \{ \dots \% \}$):** Header files are included, and global variables are declared to maintain counts of different token types (total tokens, keywords, operators, etc.).
2. **Patterns Section:** Regular expressions are defined for digits, letters, identifiers, and various constant types.

3. **Rules Section (%%...%%):** Each pattern is associated with a C action. When a pattern matches the input text (`yytext`), the corresponding action prints the token type and lexeme to the output file and increments the relevant counter.
4. **Main Function:** The program opens an input file (`input.txt`) and an output file (`output.txt`). It calls `yylex()` to begin the scanning process. Finally, it prints a summary of token counts to the output file.

4 Implementation

The Lex source code (210041226_2B.Lab02.1) is provided below:

```

1  %{
2      #include <stdio.h>
3      #include <stdlib.h>
4
5      FILE *yyin, *yyout;
6
7      int total = 0;
8      int keywords = 0;
9      int identifiers = 0;
10     int int_constants = 0;
11     int float_constants = 0;
12     int char_constants = 0;
13     int string_constants = 0;
14     int arithmetic_operators = 0;
15     int increment_decrement_operators = 0;
16     int relational_operators = 0;
17     int logical_operators = 0;
18     int bitwise_operators = 0;
19     int operators = 0;
20     int delimiters = 0;
21 %}
22
23 DIGIT      [0-9]
24 LETTER     [A-Za-z_]
25 ID         {LETTER}({LETTER}|{DIGIT})*
26 INT        {DIGIT}+
27 FLOAT      {DIGIT}+"."{DIGIT}+
28 CHAR       \'[^\']*\'
29 STRING     \"[^\"]*,[:]=*\"
30
31 %%
32 "include"|"int"|"float"|"if"|"else"|"print"|"while"|"for"|"return"|"main"|"yyin"|"yyout
33     "|"yylex"|"fopen"|"fclose"|"fprintf" {
34     fprintf(yyout, "<KEYWORD, %s>\n", yytext);
35     keywords++;
36     total++;
37 }
38 {FLOAT} {
39     fprintf(yyout, "<FLOAT_CONSTANT, %s>\n", yytext);
40     float_constants++;
41     total++;
42 }
43 {INT} {
44     fprintf(yyout, "<INTEGER_CONSTANT, %s>\n", yytext);
45     int_constants++;
46     total++;
47 }
48 {CHAR} {
49     fprintf(yyout, "<CHARACTER_CONSTANT, %s>\n", yytext);
50     char_constants++;
51     total++;
52 }
53 {STRING} {
54     fprintf(yyout, "<STRING_CONSTANT, %s>\n", yytext);
55     string_constants++;
56     total++;
57 }
58 {ID} {

```

```

59     fprintf(yyout, "<IDENTIFIER, %s>\n", yytext);
60     identifiers++;
61     total++;
62 }
63 }
64 "++"|"--" {
65     fprintf(yyout, "<INCREMENT_DECREMENT_OPERATOR, %s>\n", yytext);
66     increment_decrement_operators++;
67     operators++;
68     total++;
69 }
70 "+"|"-"|"*"|" "/"|"=" {
71     fprintf(yyout, "<ARITHMETIC_OPERATOR, %s>\n", yytext);
72     arithmetic_operators++;
73     operators++;
74     total++;
75 }
76 }
77 "=="|"!="|"<="|">="|"<"|>" {
78     fprintf(yyout, "<RELATIONAL_OPERATOR, %s>\n", yytext);
79     relational_operators++;
80     operators++;
81     total++;
82 }
83 "&&"|"||" {
84     fprintf(yyout, "<LOGICAL_OPERATOR, %s>\n", yytext);
85     logical_operators++;
86     operators++;
87     total++;
88 }
89 "&"|"^"|"~"|"<<"|>>" {
90     fprintf(yyout, "<BITWISE_OPERATOR, %s>\n", yytext);
91     bitwise_operators++;
92     operators++;
93     total++;
94 }
95 "("|")"|"{"|"}"|"["|"]"|"|"|"|"%"|"%" {
96     fprintf(yyout, "<DELIMITER, %s>\n", yytext);
97     delimiters++;
98     total++;
99 }
100 }
101 [ \t\n]+ {
102 }
103
104 %%
105 int main(){
106     yyin = fopen("input.txt", "r");
107     yyout = fopen("output.txt", "w");
108
109     yylex();
110
111     fprintf(yyout, "\nTotal Tokens: %d\n", total);
112     fprintf(yyout, "Keywords: %d\n", keywords);
113     fprintf(yyout, "Identifiers: %d\n", identifiers);
114     fprintf(yyout, "Integer Constants: %d\n", int_constants);
115     fprintf(yyout, "Float Constants: %d\n", float_constants);
116     fprintf(yyout, "Character Constants: %d\n", char_constants);
117     fprintf(yyout, "String Constants: %d\n", string_constants);
118     fprintf(yyout, "Arithmetic Operators: %d\n", arithmetic_operators);
119     fprintf(yyout, "Increment/Decrement Operators: %d\n", increment_decrement_operators);
120     fprintf(yyout, "Relational Operators: %d\n", relational_operators);
121     fprintf(yyout, "Logical Operators: %d\n", logical_operators);
122     fprintf(yyout, "Bitwise Operators: %d\n", bitwise_operators);
123     fprintf(yyout, "Delimiters: %d\n", delimiters);
124
125     fclose(yyin);
126     fclose(yyout);
127
128     return 0;

```

5 Sample Input and Output

5.1 Input (input.txt)

The scanner was tested on its own source code as input.

5.2 Output Summary (output.txt)

```
1 Total Tokens: 749
2 Keywords: 80
3 Identifiers: 182
4 Integer Constants: 17
5 Float Constants: 0
6 Character Constants: 0
7 String Constants: 48
8 Arithmetic Operators: 29
9 Increment/Decrement Operators: 29
10 Relational Operators: 34
11 Logical Operators: 0
12 Bitwise Operators: 35
13 Delimiters: 295
```

5.3 Output Log (Snippet)

```
1 <DELIMITER, %>
2 <DELIMITER, {>
3 #<KEYWORD, include>
4 <RELATIONAL_OPERATOR, <>
5 <IDENTIFIER, stdio>
6 .<IDENTIFIER, h>
7 <RELATIONAL_OPERATOR, >>
8 #<KEYWORD, include>
9 ...
10 <KEYWORD, int>
11 <IDENTIFIER, total>
12 <ARITHMETIC_OPERATOR, ==>
13 <INTEGER_CONSTANT, 0>
14 <DELIMITER, ;>
```

6 Conclusion

The lexical analyzer implemented using Lex successfully identifies and counts various token categories. This approach is highly efficient as it leverages regular expressions and automated code generation, reducing the manual effort required in scanner development.